Step 12 — Integration endpoints

What it does, and how to test it on your Docker stack step by step with the commands

Step	Phase 3, step 12: asynchronous search, signed webhooks, usage per API client
Status	Done 2026-10-23, merged into the project branch (PR #9, #10); tested by the owner 2026-10-24
Branch	`claude/cloud-vs-local-7xi988`, folder `Owis_Find_Deal_Engine/server`
New endpoints	`POST /api/v1/searches` · `GET /api/v1/searches/{id}` · `POST /admin/v1/clients/{id}/webhook-secret`
New setting	`WEBHOOK_ALLOW_PRIVATE` (.env, development only, default false)
Database	migration 0005: webhook salt, prefix and time on `api_clients` (automatic)
Date	24 October 2026

Updated after the owner's test on the Docker stack (24 Oct 2026): steps 0–13, 15 and 16 passed. Corrections from that session are in steps 1, 2, 6–11, 13, 15 and 16, and in the tips on the first page of Part 2.

> **How to read this guide.** Part 1 explains what step 12 does. Part 2 is the test: steps 0–17 with the commands to type (Git Bash on Windows) and what you should see. Only step 3 and the optional step 17 run a live search (about $0.16 Apify + 1 SerpApi search each); everything else uses the cached answer and is free.

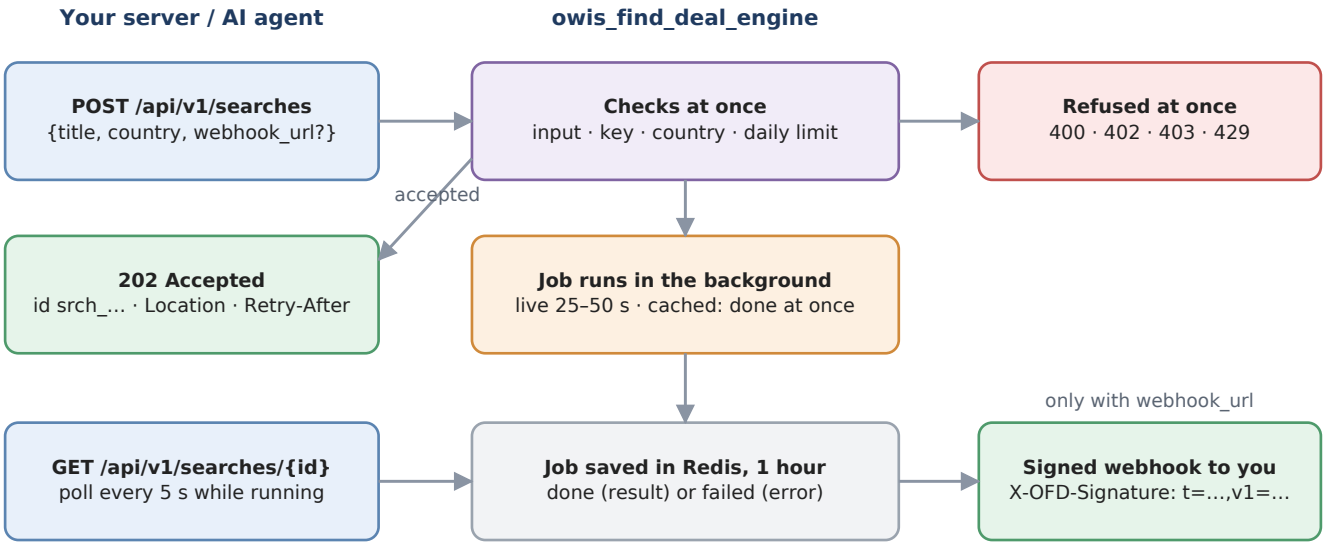
Part 1 — What step 12 does

Step 12 gives your own servers and AI agents three things they need to build on the search service.

1. Asynchronous search (start now, collect later)

A live search takes 25–50 seconds, and many programs and AI agents do not want to hold a connection open that long. Step 12 adds a second way to search:

Call	What happens
`POST /api/v1/searches` {"title", "country"}	The search starts in the background. The answer comes at once: **202 Accepted** with a job id (`srch_…`) and a `Location` header pointing to the job.
`GET /api/v1/searches/{id}`	Shows the job: **running**, **done** (with the same results as /search) or **failed** (with the error /search would have given).



Asynchronous search: the request is checked at once, the search runs in the background, the result is polled or pushed.

The rules and limits are the same as for /search:

- **It works with an API key or a user's login token.**
- **Cached answers are free.** A live search counts once against the daily limit.
- **Problems are refused immediately, before a job starts:** a bad title (400), an unknown country (400), a country the key is not allowed (403), a missing scope (403), or a used-up daily limit (402 for app users, 429 for keys).
- **Cached searches are already done** in the 202 answer, with the results included.
- **Only the caller who started a job can read it.** Anyone else gets 404.
- **Finished jobs are kept for 1 hour** in Redis, so they survive a server restart.
- **Each caller can have at most 10 jobs running at once.** The 11th gets 429 `too_many_jobs`.
- **Reading a job has its own limit** (5 per second), so polling does not use up your search limit. While a job is running, the answer includes `Retry-After: 5` ("ask again in 5 seconds").
- **If the server stops during a job**, the job ends as failed (503) and you start it again.

2. Signed webhook (the service calls you when the search is done)

This is for API keys only. Instead of polling, add `"webhook_url": "https://your-server/…"` to the request. When the job ends, the service sends the finished job to that URL with these headers:

```
X-OFD-Event:     search.completed   (or search.failed)
X-OFD-Delivery:  srch_...           (the job id: use it to ignore repeats)
X-OFD-Signature: t=<unix time>,v1=<HMAC-SHA256 of "<t>.<body>" with the client's secret>
```

- **The signature lets your receiver check** that the call really came from your service and was not changed on the way.
- **Any 2xx answer counts as delivered.** Network errors, 408, 429 and 5xx answers are retried after 5 s and again after 30 s, so 3 attempts in all. Redirects are not followed.
- **The job shows how delivery went:** `webhook.status` is pending, delivered or failed, with the number of attempts and the last answer.
- **Only https URLs to public addresses are allowed.** URLs pointing at localhost, 10.x, 192.168.x, 169.254.x and similar are refused, so nobody can make your server call its own Redis or Postgres. For a test receiver on your PC, set `WEBHOOK_ALLOW_PRIVATE=true` in .env (development only).

The signing secret is made per API client with the admin panel's API clients → Webhook secret button:

- It looks like `whsec_…` and is shown once.
- It is not stored anywhere; the server recalculates it from `SETTINGS_ENCRYPTION_KEY`.
- Making a new one stops the old one immediately.
- Each new secret is recorded in the audit log as `clients.webhook_secret`, without the secret itself.

3. Usage per API client on the dashboard

The dashboard has a new **API clients** table. For each client it shows:

- searches, cached answers, live searches, refused searches and failed searches
- **estimated cost**: the client's live searches × the average cost of a live search in the period
- the last day it searched

The same data is in `GET /admin/v1/usage` under `clients`. This is the base for billing partners later.

Error codes added by step 12

Status	code	Meaning
202	—	job started (running) or already done (cached answer)
400	webhook_not_configured	webhook_url given, but the API client has no webhook secret yet
400	invalid_request	bad input; also an unsafe webhook URL, or webhook_url with a user token
404	not_found	no such job, someone else's job, or finished more than 1 hour ago
429	too_many_jobs	10 asynchronous searches already running for this caller
job error 504	timeout	the search took too long
job error 503	job_lost / service_unavailable	the server stopped during the search: start it again

Part 2 — Test steps

The commands are for **Git Bash** on Windows. In PowerShell, type `curl.exe` instead of `curl` and put the JSON body in single quotes. Replace every <…> with your own value.

Git Bash tips (from the owner test, 24 Oct 2026).
• Run every command in the `Owis_Find_Deal_Engine/server` folder: `docker compose` and `.env` only work there (otherwise: no configuration file provided).
• `export NAME=value` with **no spaces** around = (otherwise: not a valid identifier).
• Variables live in one window only. Check them with `echo "${KEY:0:12}"`; an empty line means: export it again.
• Login tokens expire after **1 hour** (invalid or expired token); missing bearer token means the variable is empty.
• Copy job ids (`srch_…`) from the answer by hand into `export ID=…`.
• Never paste a long JSON answer into the terminal to save it: lines over ~4,000 characters are cut. Save answers with `curl … > file` instead.

Cost: step 3 runs one live search (about $0.16 Apify plus one SerpApi search). After that, the same search is cached for 48 h and every other step is free, except the optional step 17.

Step 0 — Update and start

```
cd claude-ai-clould-repo
git checkout claude/cloud-vs-local-7xi988
git pull
cd Owis_Find_Deal_Engine/server
docker compose up -d --build
docker compose ps                      # all four containers "healthy" after ~30 s
docker compose logs server | grep -i -E "migrat|error"
curl localhost:3002/api/v1/health      # {"status":"ok"}
```

✓ **Expected:** no ERROR lines; the new database migration (0005_webhooks.sql) runs automatically at start; health answers {"status":"ok"}.

Step 1 — Get tokens

Admin token (your admin email from ADMIN_EMAIL):

```
curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=<WEB_API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"email":"<admin email>","password":"<password>","returnSecureToken":true}'
export ADMIN_TOKEN=<idToken from the answer>
```

Copy the long "idToken" value (without the quotes) after ADMIN_TOKEN=, then check: `echo "${ADMIN_TOKEN:0:20}"` prints eyJhbGciOiJSUzI1NiIs.

User token (a normal app user). The email must exist as a user in **your** Firebase project; if sign-in answers EMAIL_NOT_FOUND or INVALID_LOGIN_CREDENTIALS, create the user once with `accounts:signUp` (same body):

```
curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=<WEB_API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"email":"test1@example.com","password":"Test-pass-123","returnSecureToken":true}'
export TOKEN=<idToken>
```

Next time use `accounts:signInWithPassword` for it. Quicker: any Firebase login also works as a normal app user, so `export TOKEN=$ADMIN_TOKEN` is fine for this test.

Both tokens are valid for 1 hour. Repeat this step if you later get `401 invalid or expired token`.

Step 2 — Make a test client for step 12

An **API client** is the record that owns an API key (`ofd_live_…`) and its settings: live searches per day, countries, and later its webhook secret. Your step-11 client allows only 2 live searches per day, so make a fresh one for these tests.

Admin panel: API clients → box **New client** at the top: Name `step12 agent`, Live searches per day `20`, tick **jor**, leave the rest → **Create client and key**. The box Key for "step12 agent" — copy it now appears at the top of the page: **Copy**, then **I have saved it**. The client id is the grey **#number** under its name in the Clients table. (Do not use **Edit** on an old client for this: editing makes no new key.) **Or by command:**

```
curl -s -X POST localhost:3002/admin/v1/clients \
  -H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"step12 agent","daily_quota":20,"countries":["jor"]}'
```

✓ **Expected:** status 201; the answer has `"key":"ofd_live_…"` and `"client":{"id":<N>,…,"webhook_secret_prefix":""}`.

```
export KEY=ofd_live_<...>
export CID=<N>          # the client id from the answer
```

Missed the key? It cannot be shown again (only its hash is stored): click **New key** on the client, enter **0** hours, and copy the key from the box at the top of the page.

Step 3 — A slow live search, started asynchronously (costs one live search)

```
curl -i -X POST localhost:3002/api/v1/searches \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"title":"usb c hub","country":"jor"}'
```

✓ **Expected:** the answer comes **at once**:

- `HTTP/1.1 202 Accepted`
- `Location: /api/v1/searches/srch_…`
- `Retry-After: 5` and `X-Plan: api`
- body `{"id":"srch_…","status":"running",…,"finished_at":null}`

Save the id, then poll every 5 seconds until the job is no longer running:

```
export ID=srch_<...>
while true; do
  S=$(curl -s localhost:3002/api/v1/searches/$ID -H "Authorization: Bearer $KEY")
  echo "$S" | grep -o '"status":"[a-z]*"' | head -1
  echo "$S" | grep -q '"status":"running"' || break
  sleep 5
done
echo "$S"
```

✓ **Expected:** "status":"running" several times (about 30–50 s), then "status":"done". The final answer has:

- `"result":{…}` with the products and `"cached":false`
- `"finished_at"` with a time
- `"quota":{"plan":"api","limit":20,"remaining":19,…}`

Step 4 — A cached search is done immediately (free)

```
curl -i -X POST localhost:3002/api/v1/searches \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"title":"usb c hub","country":"jor"}'
```

✓ **Expected:** 202 with "status":"done" and "cached":true in the body. There is **no** Retry-After, and X-RateLimit-Remaining is still 19, because cached answers are free.

Step 5 — A user login token works too

```
curl -i -X POST localhost:3002/api/v1/searches \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"title":"usb c hub","country":"jor"}'
```

✓ **Expected:** 202, "status":"done", X-Plan: free.

Step 6 — Refusals happen immediately (free)

```
# title too short -> 400 invalid_request
curl -s -X POST localhost:3002/api/v1/searches -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -d '{"title":"x","country":"jor"}'
# made-up country -> 403 country_not_allowed (this key may search jor only)
curl -s -X POST localhost:3002/api/v1/searches -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -d '{"title":"usb c hub","country":"xyz"}'
# country not allowed for this key -> 403 country_not_allowed
curl -s -X POST localhost:3002/api/v1/searches -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -d '{"title":"usb c hub","country":"usa"}'
# no token -> 401
curl -s -X POST localhost:3002/api/v1/searches \
  -H "Content-Type: application/json" -d '{"title":"usb c hub","country":"jor"}'
# made-up field -> 400 invalid_request ("unknown field")
curl -s -X POST localhost:3002/api/v1/searches -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" -d '{"title":"usb c hub","country":"jor","x":1}'
```

✓ **Expected:** each answers right away with `{"error":{"code":"…","message":"…"}}` and the code shown in the comment (put `;  echo` after each command to get one answer per line).

A key limited to some countries answers `country_not_allowed` for every other code, real or made-up: that check comes first, as on POST /api/v1/search. `400 unsupported_country` comes only with a key that allows every country, or with a user token.

Step 7 — Nobody else can read the job

```
# the user token reading the key's job -> 404
curl -s localhost:3002/api/v1/searches/$ID -H "Authorization: Bearer $TOKEN"
# a made-up id -> 404
curl -s localhost:3002/api/v1/searches/srch_aaaaaaaaaaaaaaaaaaaaaaaa \
  -H "Authorization: Bearer $KEY"
```

✓ **Expected:** both answer 404 not_found ("no search job with this id…").

Jobs are kept 1 hour after they finish. To be sure the first 404 is the owner check and not an expired job, read the same job with $KEY: it must still show it. If it has expired, start a new (cached, free) search with the key, export its id and repeat.

Step 8 — The job survives a server restart (stored in Redis)

```
docker compose restart server
sleep 15
curl -s localhost:3002/api/v1/searches/$ID -H "Authorization: Bearer $KEY" | head -c 300
```

✓ **Expected:** the job from step 3 is still there with "status":"done", as long as you are within 1 hour of when it finished.

A plain-text `404 page not found` (not JSON) means `$ID` is empty: check `echo "id: $ID"` and export it again.

Step 9 — Webhook without a secret is refused

Go to **https://webhook.site**, which gives you a free https URL such as `https://webhook.site/1234-…` ("Your unique URL"). Keep the tab open and put your URL into the body:

```
curl -s -X POST localhost:3002/api/v1/searches \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"title":"usb c hub","country":"jor","webhook_url":"https://webhook.site/<your id>"}'; echo
export HOOK=https://webhook.site/<your id>      # used by step 12
```

✓ **Expected:** 400 with "code":"webhook_not_configured".

Step 10 — Make the webhook secret

Use the admin panel (**API clients** → **Webhook secret** on step12 agent → copy it) or:

```
curl -s -X POST localhost:3002/admin/v1/clients/$CID/webhook-secret \
  -H "Authorization: Bearer $ADMIN_TOKEN"
```

✓ **Expected:** 201 with `"webhook_secret":"whsec_…"`. The client list now shows `whsec_xxxxxxxx…` in the **Webhook** column.

```
export SECRET=whsec_<...>
```

The server may take up to 10 s to see the new secret, so wait about 10 s before step 11. `invalid or expired token`: the admin token is older than 1 hour, get a new one (step 1) or use the button in the admin panel, which needs no token.

Step 11 — Webhook delivered and signed (free, uses the cached search)

```
curl -s -X POST localhost:3002/api/v1/searches \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"title":"usb c hub","country":"jor","webhook_url":"https://webhook.site/<your id>"}' | head -c 120; echo
```

✓ **Expected:** `{"id":"srch_…","status":"done",…` (cached, so done at once). Copy the id: `export ID2=srch_…`

✓ **Expected:** within a few seconds webhook.site lists a new **POST** on the left. Click it: the **Headers** table (next to or below Request Details; header names in lower case) shows `x-ofd-event: search.completed`, `x-ofd-delivery: srch_…` (the same id) and `x-ofd-signature: t=…,v1=…`; **Raw Content** is the job JSON. Can't find it? Ctrl+F for `ofd`.

What the service recorded:

```
curl -s localhost:3002/api/v1/searches/$ID2 -H "Authorization: Bearer $KEY" \
  | grep -o '"webhook":{[^}]*}'
```

✓ **Expected:** `"webhook":{…"status":"delivered","attempts":1,"http_status":200}` (still `pending`? wait 5 s and repeat).

Check the signature. Create `verify.py` (paste the whole block):

```
cat > verify.py <<'EOF'
import hashlib, hmac, sys, time
secret, header, body_file = sys.argv[1], sys.argv[2].strip(), sys.argv[3]
body = open(body_file, "rb").read().rstrip(b"\r\n")
parts = dict(p.split("=", 1) for p in header.split(","))
want = hmac.new(secret.encode(), parts["t"].encode() + b"." + body, hashlib.sha256).hexdigest()
print("signature OK" if hmac.compare_digest(want, parts["v1"]) else "BAD signature")
print("age:", int(time.time()) - int(parts["t"]), "s")
EOF
```

The webhook body is exactly the job as GET returns it, without its `"webhook":{…}` part (that delivery state did not exist yet when it was sent). Rebuild it byte for byte from the server (do **not** paste webhook.site's Raw Content into the terminal: the line is ~8,000 characters and gets cut, which gives BAD signature). Copy the x-ofd-signature value from webhook.site (without a leading tab):

```
curl -s localhost:3002/api/v1/searches/$ID2 -H "Authorization: Bearer $KEY" \
  | sed 's/,"webhook":{[^}]*}//' > body.json
wc -c body.json                                  # about 7,700 bytes
python verify.py "$SECRET" "t=...,v1=..." body.json
sed -i 's/"status":"done"/"status":"dona"/' body.json    # change one character
python verify.py "$SECRET" "t=...,v1=..." body.json
```

✓ **Expected:** first `signature OK`, then `BAD signature` after the change. (`py` instead of `python` if that is not found. The age printed is seconds since sending; a real receiver refuses more than 5 minutes.)

Step 12 — Unsafe webhook URLs are refused (free)

```
for U in http://example.com/hook https://127.0.0.1/hook \
         https://192.168.1.10/hook https://localhost/hook; do
  curl -s -X POST localhost:3002/api/v1/searches -H "Authorization: Bearer $KEY" \
    -H "Content-Type: application/json" \
    -d "{\"title\":\"usb c hub\",\"country\":\"jor\",\"webhook_url\":\"$U\"}"; echo
done
# webhooks are for API keys only -> 400
curl -s -X POST localhost:3002/api/v1/searches -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d "{\"title\":\"usb c hub\",\"country\":\"jor\",\"webhook_url\":\"$HOOK\"}"
```

✓ **Expected:** all answer 400 invalid_request:

- the http URL gets "must use https";
- the three internal addresses get "must not point to an internal address";
- the user-token request gets "webhook_url is only for API keys".

Step 13 — A new secret replaces the old one (free)

Keep the old secret, make a new one (or click **Webhook secret** again in the admin panel):

```
export OLD_SECRET=$SECRET
curl -s -X POST localhost:3002/admin/v1/clients/$CID/webhook-secret \
  -H "Authorization: Bearer $ADMIN_TOKEN" | grep -o '"webhook_secret":"[^"]*"'; echo
export SECRET=whsec_<the new secret>
```

Wait 10 s, run the first command of step 11 again, export the new id as `ID3` and copy the **new** x-ofd-signature from webhook.site (the newest POST; its x-ofd-delivery is `$ID3`):

```
curl -s localhost:3002/api/v1/searches/$ID3 -H "Authorization: Bearer $KEY" \
  | sed 's/,"webhook":{[^}]*}//' > body3.json
python verify.py "$OLD_SECRET" "<new signature>" body3.json
python verify.py "$SECRET"     "<new signature>" body3.json
```

✓ **Expected:** first `BAD signature` (the old secret no longer signs), then `signature OK`.

Step 14 (optional) — Receiver on your PC: failures and retries (free)

This step uses a receiver script that checks every webhook it receives.

1. Add `WEBHOOK_ALLOW_PRIVATE=true` to .env, then run `docker compose up -d` again.
2. Save this as `receiver.py`:

```
import hashlib, hmac, sys, time
from http.server import BaseHTTPRequestHandler, HTTPServer
SECRET = sys.argv[1]
class H(BaseHTTPRequestHandler):
    def do_POST(self):
        body = self.rfile.read(int(self.headers["Content-Length"]))
        parts = dict(p.split("=", 1) for p in self.headers["X-OFD-Signature"].split(","))
        want = hmac.new(SECRET.encode(), parts["t"].encode() + b"." + body, hashlib.sha256).hexdigest()
        ok = hmac.compare_digest(want, parts["v1"]) and abs(time.time() - int(parts["t"])) < 300
        print(self.headers["X-OFD-Event"], self.headers["X-OFD-Delivery"],
              "OK" if ok else "BAD SIGNATURE", flush=True)
        self.send_response(204 if ok else 401)
        self.end_headers()
HTTPServer(("0.0.0.0", 9000), H).serve_forever()
```

3. Run the three cases below. Each time, start the search with `"webhook_url":"http://host.docker.internal:9000/hook"` and read the job afterwards.

Case	How	Expected
Correct secret	`python receiver.py "$SECRET"`	The receiver prints `search.completed srch_…` OK. The job shows delivered, http_status 204.
Wrong secret	`python receiver.py whsec_wrong`	The receiver prints BAD SIGNATURE and answers 401. The job shows `"status":"failed","attempts":1,"http_status":401`. A 4xx answer is not retried.
Receiver stopped	Stop the script (Ctrl+C)	After about 35 s the job shows `"status":"failed","attempts":3` with "connection refused". The server log has a webhook not delivered warning.

4. When you are done, remove WEBHOOK_ALLOW_PRIVATE from .env and run `docker compose up -d` again.

Step 15 — Dashboard per client (free)

Wait about 30 s, because the counts are saved every 30 s. Then:

- **Admin panel → Dashboard → API clients table:** step12 agent shows live 1 and the cached searches, plus an estimated cost and today's date. **Searches** counts only searches that ran (live or cached); requests refused before a job starts (steps 6, 9, 12) are not counted.
- Or with a command:

```
curl -s "localhost:3002/admin/v1/usage?days=1" -H "Authorization: Bearer $ADMIN_TOKEN" \
  | grep -o '"clients":\[[^]]*\]'
```

✓ **Expected:** a row with "name":"step12 agent", "live":1, "cost_usd_estimate":….

`cost_usd_estimate` is 0 until the cost settings are set (admin panel → Settings): `APIFY_ALIEXPRESS_RUN_COST_USD` 0.02, `APIFY_TEMU_RUN_COST_USD` 0.10, `APIFY_SHEIN_RUN_COST_USD` 0.033, `SERPAPI_SEARCH_COST_USD` (0 on the free plan). They count from then on.

Step 16 — Audit log and logs never contain the secret (free)

```
curl -s "localhost:3002/admin/v1/audit?action=clients.webhook_secret" \
  -H "Authorization: Bearer $ADMIN_TOKEN"
cd <repo>/Owis_Find_Deal_Engine/server         # docker compose works only here
docker compose logs server | grep -c "whsec_"      # must be 0
docker compose logs server | grep -c "ofd_live_"   # must be 0
```

✓ **Expected:** the audit entries show only `"prefix":"whsec_xxxxxxxx"` (and "replaced" after step 13). Both counts are 0.

Step 17 (optional) — Daily limit refused before a job starts (costs one live search)

1. Make a client with `"daily_quota":1`.
2. Run one live search with a **new** title through it.
3. Start a second **new**, uncached title with POST /api/v1/searches.

✓ **Expected:** the second request answers 429 `quota_exceeded` immediately, with Retry-After and X-RateLimit-Remaining: 0. No job is created.

What to report back

1. Which steps passed or failed, with the status code and `docker compose logs server` lines for any failure.
2. Whether polling every 5 s suits your agents.
3. Whether one webhook secret per client is enough, or you want a fixed webhook URL per client set in the admin panel.